Four-Wheel Robot Simulation

A Complete ROS 2 & Gazebo Tutorial

URDF from Scratch · Package Setup · Gazebo Classic
ROS 2 Humble · Ubuntu 22.04

Covers: package creation, URDF geometry, joint types, diff_drive plugin, launch files, teleoperation

Last updated: April 5, 2026

Contents

1 Overview **2**

2 Phase 1 — Install Required Packages **2**
- 2.1 Gazebo Classic and ROS 2 Bridge 2
- 2.2 Verify the Installation 3

3 Phase 2 — Create the ROS 2 Package **3**
- 3.1 Package Creation 3
- 3.2 CMakeLists.txt 3
- 3.3 package.xml 4

4 Phase 3 — Write the URDF **4**
- 4.1 Materials 5
- 4.2 Base Link (Chassis) 5
- 4.3 Front Wheels (Fixed — Passive) 5
- 4.4 Rear Wheels (Continuous — Driven) 7
- 4.5 Joint Origin Explained 8
- 4.6 Lidar Sensor 8

5 Phase 4 — Gazebo Properties and Plugin **9**
- 5.1 Surface Friction Tags 9
- 5.2 Differential Drive Plugin 9

6 Phase 5 — World File **10**

7 Phase 6 — Launch File **11**

8 Phase 7 — Build and Launch **12**
- 8.1 Validate the URDF First 12
- 8.2 Build the Package 12
- 8.3 Verify Install Directory 13
- 8.4 Launch 13

9 Phase 8 — Drive the Robot **13**
- 9.1 Publish Velocity Commands Directly 13
- 9.2 Keyboard Teleoperation 13
- 9.3 Monitor Topics 14

10 Troubleshooting **14**

11 File Summary **15**

12 Next Steps **15**

1 Overview

This tutorial walks through building a four-wheel differential drive robot completely from scratch using ROS 2 Humble and Gazebo Classic (version 11). By the end you will have:

- A working ROS 2 package with the correct folder structure
- A complete URDF file describing a four-wheel robot with a chassis, four wheels, and a lidar sensor
- Gazebo friction and surface properties for realistic physics
- A `diff_drive` plugin that accepts `/cmd_vel` velocity commands
- A Python launch file that starts Gazebo and spawns the robot
- Working teleoperation from the keyboard

> **Prerequisites:** ROS 2 Humble installed on Ubuntu 22.04, a configured workspace at `~/ros2_ws`, and basic familiarity with the terminal.

Robot Dimensions at a Glance

Part	Geometry	Size
Chassis	Box	0.5 m × 0.4 m × 0.1 m
Wheels (4)	Cylinder	radius 0.1 m, length 0.05 m
Lidar	Cylinder	radius 0.05 m, length 0.04 m

Drive Architecture

The `libgazebo_ros_diff_drive.so` plugin in ROS 2 Humble accepts **exactly one left joint and one right joint**. The rear wheels are therefore `continuous` (driven), while the front wheels are `fixed` (passive). This is the standard approach for four-wheel diff-drive robots in Gazebo Classic.

2 Phase 1 — Install Required Packages

2.1 Gazebo Classic and ROS 2 Bridge

```
sudo apt update

# Gazebo Classic (version 11)     the version paired with ROS 2
   Humble
sudo apt install -y gazebo

# ROS 2 <-> Gazebo bridge (provides gazebo_ros package)
sudo apt install -y ros-humble-gazebo-ros-pkgs

# State publishers
sudo apt install -y ros-humble-robot-state-publisher
sudo apt install -y ros-humble-joint-state-publisher
```

```
sudo apt install -y ros-humble-joint-state-publisher-gui

# Xacro (needed by launch files even when using plain URDF)
sudo apt install -y ros-humble-xacro
```

2.2 Verify the Installation

```
source /opt/ros/humble/setup.bash

# Should print the gazebo_ros install path
ros2 pkg prefix gazebo_ros

# Should print "Gazebo multi-robot simulator, version 11.x.x"
gazebo --version
```

> If both commands return output without errors, the installation is complete.

3 Phase 2 — Create the ROS 2 Package

3.1 Package Creation

We use an `ament_cmake` package (not Python) because robot description packages contain only configuration files — no executable nodes.

```
cd ~/ros2_ws/src

ros2 pkg create four_wheel_robot \
  --build-type ament_cmake \
  --dependencies rclcpp

cd four_wheel_robot
mkdir urdf launch worlds rviz
```

The resulting folder structure:

```
four_wheel_robot/
          CMakeLists.txt
          package.xml
          urdf/           # URDF robot description
          launch/         # Python launch files
          worlds/         # Gazebo world SDF files
          rviz/           # Optional RViz configs
```

3.2 CMakeLists.txt

Replace the contents of `CMakeLists.txt` with:

```
cmake_minimum_required(VERSION 3.8)
project(four_wheel_robot)
```

```
find_package(ament_cmake REQUIRED)

# Install all resource folders into the package share directory
install(DIRECTORY
  urdf
  launch
  worlds
  rviz
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

> Every folder listed in `install(DIRECTORY ...)` must physically exist before building, otherwise `colcon build` will warn about missing paths. The `mkdir` command in the previous step creates them.

3.3 package.xml

Replace the contents of `package.xml` with:

```
<?xml version="1.0"?>
<package format="3">
  <name>four_wheel_robot</name>
  <version>0.0.1</version>
  <description>Four-wheel robot URDF and Gazebo
     simulation</description>
  <maintainer email="you@email.com">Your Name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <exec_depend>robot_state_publisher</exec_depend>
  <exec_depend>joint_state_publisher</exec_depend>
  <exec_depend>gazebo_ros</exec_depend>
  <exec_depend>xacro</exec_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

4 Phase 3 — Write the URDF

Create the file `urdf/four_wheel_robot.urdf`:

```
nano ~/ros2_ws/src/four_wheel_robot/urdf/four_wheel_robot.urdf
```

Paste the complete URDF below. Each section is explained afterward.

4.1 Materials

```
<?xml version="1.0"?>
<robot name="four_wheel_robot">

  <material name="blue">
    <color rgba="0.2 0.4 0.8 1.0"/>
  </material>
  <material name="dark_gray">
    <color rgba="0.15 0.15 0.15 1.0"/>
  </material>
  <material name="black">
    <color rgba="0.05 0.05 0.05 1.0"/>
  </material>
```

4.2 Base Link (Chassis)

```
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry><box size="0.5 0.4 0.1"/></geometry>
      <material name="blue"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry><box size="0.5 0.4 0.1"/></geometry>
    </collision>
    <inertial>
      <mass value="4.0"/>
      <inertia ixx="0.0587" ixy="0" ixz="0"
               iyy="0.0917" iyz="0" izz="0.1333"/>
    </inertial>
  </link>
```

Inertia formula for a box with mass m, dimensions $x \times y \times z$:

$$I_{xx} = \frac{m}{12}(y^2 + z^2), \quad I_{yy} = \frac{m}{12}(x^2 + z^2), \quad I_{zz} = \frac{m}{12}(x^2 + y^2)$$

4.3 Front Wheels (Fixed — Passive)

The front wheels use `type="fixed"` joints because the `diff_drive` plugin only accepts one joint per side. They are physically present but not actuated.

```
  <!-- FRONT LEFT WHEEL -->
  <link name="front_left_wheel">
    <visual>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
```

```
      <geometry><cylinder radius="0.1"
         length="0.05"/></geometry>
      <material name="dark_gray"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1"
         length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.00052" ixy="0" ixz="0"
               iyy="0.00052" iyz="0" izz="0.00100"/>
    </inertial>
  </link>

  <joint name="front_left_wheel_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="front_left_wheel"/>
    <origin xyz="0.15 0.225 -0.05" rpy="0 0 0"/>
  </joint>

  <!-- FRONT RIGHT WHEEL -->
  <link name="front_right_wheel">
    <visual>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1"
         length="0.05"/></geometry>
      <material name="dark_gray"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry><cylinder radius="0.1"
         length="0.05"/></geometry>
    </collision>
    <inertial>
      <mass value="0.5"/>
      <inertia ixx="0.00052" ixy="0" ixz="0"
               iyy="0.00052" iyz="0" izz="0.00100"/>
    </inertial>
  </link>

  <joint name="front_right_wheel_joint" type="fixed">
    <parent link="base_link"/>
    <child  link="front_right_wheel"/>
    <origin xyz="0.15 -0.225 -0.05" rpy="0 0 0"/>
  </joint>
```

4.4 Rear Wheels (Continuous — Driven)

The rear wheels use `type="continuous"` joints. These are the joints the `diff_drive` plugin will actuate.

```
<!-- REAR LEFT WHEEL -->
<link name="rear_left_wheel">
  <visual>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.1"
       length="0.05"/></geometry>
    <material name="dark_gray"/>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.1"
       length="0.05"/></geometry>
  </collision>
  <inertial>
    <mass value="0.5"/>
    <inertia ixx="0.00052" ixy="0" ixz="0"
             iyy="0.00052" iyz="0" izz="0.00100"/>
  </inertial>
</link>

<joint name="rear_left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child  link="rear_left_wheel"/>
  <origin xyz="-0.15 0.225 -0.05" rpy="0 0 0"/>
  <axis   xyz="0 1 0"/>
</joint>

<!-- REAR RIGHT WHEEL -->
<link name="rear_right_wheel">
  <visual>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.1"
       length="0.05"/></geometry>
    <material name="dark_gray"/>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <geometry><cylinder radius="0.1"
       length="0.05"/></geometry>
  </collision>
  <inertial>
    <mass value="0.5"/>
    <inertia ixx="0.00052" ixy="0" ixz="0"
             iyy="0.00052" iyz="0" izz="0.00100"/>
  </inertial>
</link>
```

```
<joint name="rear_right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child  link="rear_right_wheel"/>
  <origin xyz="-0.15 -0.225 -0.05" rpy="0 0 0"/>
  <axis   xyz="0 1 0"/>
</joint>
```

Why `rpy="1.5708 0 0"` on the wheel visual? Cylinders in URDF are aligned along the Z-axis by default. Rotating 90° around X (`rpy="`$\pi/2$ `0 0"`) makes the cylinder lie on its side so it rolls correctly along the ground.

4.5 Joint Origin Explained

The `<origin>` on each wheel joint places the wheel relative to `base_link`:

Wheel	X (m)	Y (m)	Z (m)	Note
Front left	+0.15	+0.225	−0.05	forward, left, ground level
Front right	+0.15	−0.225	−0.05	forward, right, ground level
Rear left	−0.15	+0.225	−0.05	rear, left, ground level
Rear right	−0.15	−0.225	−0.05	rear, right, ground level

4.6 Lidar Sensor

```
<link name="lidar_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry><cylinder radius="0.05"
       length="0.04"/></geometry>
    <material name="black"/>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry><cylinder radius="0.05"
       length="0.04"/></geometry>
  </collision>
  <inertial>
    <mass value="0.1"/>
    <inertia ixx="0.0001" ixy="0" ixz="0"
             iyy="0.0001" iyz="0" izz="0.0001"/>
  </inertial>
</link>

<joint name="lidar_joint" type="fixed">
  <parent link="base_link"/>
  <child  link="lidar_link"/>
  <origin xyz="0.1 0 0.07" rpy="0 0 0"/>
</joint>
```

5 Phase 4 — Gazebo Properties and Plugin

5.1 Surface Friction Tags

```
<gazebo reference="base_link">
  <material>Gazebo/Blue</material>
</gazebo>

<gazebo reference="front_left_wheel">
  <material>Gazebo/DarkGrey</material>
  <surface>
    <friction><ode><mu>1.0</mu><mu2>1.0</mu2></ode></friction>
    <contact><ode><kp>1e6</kp><kd>100</kd></ode></contact>
  </surface>
</gazebo>

<!-- repeat identical block for front_right_wheel,
     rear_left_wheel, rear_right_wheel -->

<gazebo reference="caster_link">
  <material>Gazebo/Grey</material>
  <surface>
    <friction><ode><mu>0.01</mu><mu2>0.01</mu2></ode></friction>
  </surface>
</gazebo>

<gazebo reference="lidar_link">
  <material>Gazebo/Black</material>
</gazebo>
```

Parameter	Value	Meaning
mu / mu2 (drive wheels)	1.0	High grip — wheels propel the robot
mu / mu2 (caster)	0.01	Near-zero — slides freely
kp	1e6	Contact stiffness
kd	100	Contact damping

5.2 Differential Drive Plugin

```
<gazebo>
  <plugin name="diff_drive_controller"
          filename="libgazebo_ros_diff_drive.so">

    <update_rate>50</update_rate>

    <!-- ONE joint per side only     Humble limitation -->
    <left_joint>rear_left_wheel_joint</left_joint>
    <right_joint>rear_right_wheel_joint</right_joint>

    <!-- Must match actual geometry exactly -->
    <wheel_separation>0.45</wheel_separation>
```

```
      <wheel_diameter>0.2</wheel_diameter>

      <max_wheel_torque>20</max_wheel_torque>
      <max_wheel_acceleration>1.0</max_wheel_acceleration>

      <command_topic>cmd_vel</command_topic>
      <odometry_topic>odom</odometry_topic>
      <odometry_frame>odom</odometry_frame>
      <robot_base_frame>base_link</robot_base_frame>

      <publish_odom>true</publish_odom>
      <publish_odom_tf>true</publish_odom_tf>
      <publish_wheel_tf>false</publish_wheel_tf>

    </plugin>
  </gazebo>

</robot>
```

Critical: The `libgazebo_ros_diff_drive.so` plugin in ROS 2 Humble supports **exactly one** `<left_joint>` and one `<right_joint>`. Specifying two joints per side causes the error `Inconsistent number of joints specified. Plugin will not work.` Always use one driven joint per side.

Wheel Separation Calculation

`wheel_separation` is the distance between the two driven wheel centres along the Y-axis:

$$\text{wheel_separation} = |y_{\text{left}}| + |y_{\text{right}}| = 0.225 + 0.225 = 0.45 \text{ m}$$

How cmd_vel Steers the Robot

The plugin converts a single `Twist` message into individual wheel speeds:

$$v_{\text{left}} = v_x - \frac{\omega_z \cdot d}{2}, \qquad v_{\text{right}} = v_x + \frac{\omega_z \cdot d}{2}$$

where v_x is `linear.x`, ω_z is `angular.z`, and d is `wheel_separation`.

6 Phase 5 — World File

Create `worlds/empty.world`:

```
cat > ~/ros2_ws/src/four_wheel_robot/worlds/empty.world << 'EOF'
<?xml version="1.0"?>
<sdf version="1.6">
  <world name="default">
    <include><uri>model://sun</uri></include>
    <include><uri>model://ground_plane</uri></include>
    <physics type="ode">
      <max_step_size>0.001</max_step_size>
```

```
      <real_time_factor>1</real_time_factor>
      <real_time_update_rate>1000</real_time_update_rate>
    </physics>
  </world>
</sdf>
EOF
```

7 Phase 6 — Launch File

Create launch/gazebo.launch.py:

```
import os
from ament_index_python.packages import
   get_package_share_directory
from launch import LaunchDescription
from launch.actions import ExecuteProcess, TimerAction
from launch_ros.actions import Node


def generate_launch_description():

    pkg = get_package_share_directory('four_wheel_robot')
    urdf_file  = os.path.join(pkg, 'urdf',
       'four_wheel_robot.urdf')
    world_file = os.path.join(pkg, 'worlds', 'empty.world')

    with open(urdf_file, 'r') as f:
        robot_description = f.read()

    return LaunchDescription([

        # 1. Start Gazebo with our world
        ExecuteProcess(
            cmd=[
                'gazebo', '--verbose', world_file,
                '-s', 'libgazebo_ros_init.so',
                '-s', 'libgazebo_ros_factory.so'
            ],
            output='screen'
        ),

        # 2. Publish robot description on /robot_description
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            parameters=[{
                'robot_description': robot_description,
                'use_sim_time': True
            }],
```

```
            output='screen'
        ),

        # 3. Spawn robot     wait 3s for Gazebo to finish
            loading
        TimerAction(
            period=3.0,
            actions=[
                Node(
                    package='gazebo_ros',
                    executable='spawn_entity.py',
                    arguments=[
                        '-topic', '/robot_description',
                        '-entity', 'four_wheel_robot',
                        '-x', '0.0', '-y', '0.0', '-z', '0.15'
                    ],
                    output='screen'
                )
            ]
        ),

    ])
```

8 Phase 7 — Build and Launch

8.1 Validate the URDF First

```
source /opt/ros/humble/setup.bash
check_urdf
   ~/ros2_ws/src/four_wheel_robot/urdf/four_wheel_robot.urdf
```

Expected output:

```
robot name is: four_wheel_robot
---------- Successfully Parsed XML ---------------
root Link: base_link has 5 children
    child(1):  front_left_wheel
    child(2):  front_right_wheel
    child(3):  lidar_link
    child(4):  rear_left_wheel
    child(5):  rear_right_wheel
```

8.2 Build the Package

```
cd ~/ros2_ws

# Clean build (recommended after adding new files)
rm -rf build/four_wheel_robot install/four_wheel_robot
colcon build --packages-select four_wheel_robot
```

```
source install/setup.bash
```

8.3 Verify Install Directory

```
ls ~/ros2_ws/install/four_wheel_robot/share/four_wheel_robot/
# Should show: urdf/  launch/  worlds/  rviz/

ls
   ~/ros2_ws/install/four_wheel_robot/share/four_wheel_robot/worlds/
# Should show: empty.world
```

8.4 Launch

```
ros2 launch four_wheel_robot gazebo.launch.py
```

Gazebo opens. After approximately 3 seconds the robot drops onto the ground plane — a blue box chassis with 4 dark wheels and a small black lidar cylinder on top.

> Confirm the plugin loaded successfully by looking for this line in the terminal output:
> `[Msg] Loaded plugin: libgazebo_ros_diff_drive.so`

9 Phase 8 — Drive the Robot

9.1 Publish Velocity Commands Directly

Open a second terminal:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

# Drive forward
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.5}, angular: {z: 0.0}}" --rate 10

# Turn left while moving
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.3}, angular: {z: 0.5}}" --rate 10

# Stop
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0}, angular: {z: 0.0}}" --once
```

9.2 Keyboard Teleoperation

```
# Install teleop package
sudo apt install ros-humble-teleop-twist-keyboard
```

```
# Run (in a new terminal, with workspace sourced)
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args --remap cmd_vel:=/cmd_vel
```

Key	Action
i	Forward
,	Backward
j	Turn left
l	Turn right
k	Stop
q / z	Increase / decrease speed

9.3 Monitor Topics

```
# List all active topics
ros2 topic list
# Expected: /cmd_vel  /odom  /robot_description  /tf  /clock

# Watch odometry output
ros2 topic echo /odom

# Visualise the TF tree
ros2 run tf2_tools view_frames
evince frames.pdf
```

10 Troubleshooting

Error	Fix
`package 'gazebo_ros' not found`	`sudo apt install ros-humble-gazebo-ros-pkgs`
`Inconsistent number of joints`	Plugin only accepts one `<left_joint>` and one `<right_joint>`. Remove extra joint tags.
World file not found	Run `rm -rf build/ install/` for the package then rebuild. Check `CMakeLists.txt` lists `worlds` in `install(DIRECTORY ...)`.
Robot falls through ground	Increase spawn height: change `'-z', '0.15'` to `'-z', '0.25'` in the launch file.
`spawn_entity.py` times out	Increase `TimerAction` period from `3.0` to `5.0` seconds.
`colcon build` fails	Run `rosdep install --from-paths src --ignore-src -r -y` then rebuild.

11 File Summary

File	Purpose
`package.xml`	ROS 2 package metadata and runtime dependencies
`CMakeLists.txt`	Installs resource folders into package share directory
`urdf/four_wheel_robot.urdf`	Robot geometry, joints, Gazebo surface properties, and diff_drive plugin
`worlds/empty.world`	Gazebo SDF world with ground plane, sun, and physics settings
`launch/gazebo.launch.py`	Starts Gazebo, publishes URDF, and spawns the robot

12 Next Steps

- **Xacro** — Refactor the four near-identical wheel blocks into a single reusable `xacro:macro`, reducing the URDF from ∼200 lines to ∼80 lines.
- **Lidar plugin** — Add `libgazebo_ros_ray_sensor.so` to `lidar_link` so it publishes real `sensor_msgs/LaserScan` data on `/scan`.
- **RViz** — Launch RViz alongside Gazebo to visualise the robot model, TF tree, and laser scan data simultaneously.
- **Navigation** — Integrate `nav2` with the `/odom` and `/scan` topics for autonomous navigation.